VERI BILIMCI MUFREDATI - MODUL 20

20. ML PIPELINES

Pipeline - ColumnTransformer - Preprocessing - Reproducibility

Raw Data → **Impute** → **Encode / Scale** → **Feature Engineering** → **Model** → **Prediction**

Bu modülün amacı

Makine öğrenmesi projesini notebook hücreleri toplamı olmaktan çıkarıp; aynı preprocessing ve feature logic ile tekrar üretilebilen, cross-validation içinde leakage yaratmayan ve production inference ile aynı akışı kullanan bir sistem haline getirmek.

Kazanım	Bu modül sonunda
Pipeline mantığı	fit/transform/predict zincirini ve neden tek nesneye ihtiyaç duyulduğunu açıklayabilirsin.
ColumnTransformer	Sayısal ve kategorik sütunlara farklı preprocessing uygulayabilirsin.
Leakage kontrolü	Scaler, imputer, encoder ve feature selection adımlarını CV içinde güvenli çalıştırabilirsin.
Tuning	Preprocessing ve model hyperparameter'larını tek GridSearch/RandomizedSearch içinde optimize edebilirsin.
Reproducibility	Aynı pipeline'ı kaydedip tekrar yükleyebilir, schema ve versiyon mantığını kurabilirsin.
Production hazırlığı	Train-serving skew riskini azaltan inference akışı kurabilirsin.

1. ML Pipeline nedir?

ML pipeline, verinin ham halden model tahminine kadar geçtiği adımları belirli bir sırayla tanımlayan yürütülebilir bir akıştır. Amaç sadece kodu güzelleştirmek değildir; training ile validation ve production arasında aynı dönüşümlerin uygulanmasını garanti etmektir.

Notebook yaklaşımı	Pipeline yaklaşımı
Önce fillna, sonra scaler, sonra get_dummies, sonra model	Tüm adımlar tek nesne içinde tanımlı
Hücre sırası değişirse sonuç değişebilir	Akış deterministik ve açık
CV öncesi preprocessing leakage yaratabilir	Her fold içinde preprocessing yeniden fit edilir
Deployment için dönüşümleri yeniden yazmak gerekir	Aynı pipeline predict() için kullanılabilir
Feature isimleri ve parametreler dağınık	Tek model artifact içinde toplanabilir

Ana fikir

Pipeline = data transformation graph + estimator. Modelin parçası sadece son estimator değildir; preprocessing de model sisteminin parçasıdır.

1.1 Pipeline hangi problemleri çözer?

- Data leakage riskini azaltır.
- Cross-validation ve hyperparameter tuning'i doğru yapar.
- Train ve inference arasında aynı işlemleri uygular.
- Kodu test edilebilir ve yeniden kullanılabilir hale getirir.
- Deneylerin tekrar üretilebilirliğini artırır.

2. fit, transform, fit_transform, predict

Metot	Ne yapar?	Örnek
fit(X, y)	Training verisinden parametre öğrenir	Scaler mean/std; imputer median; model weights
transform(X)	Öğrenilmiş dönüşümü uygular	X -> scaled X
fit_transform(X)	fit + transform	Training preprocessing
predict(X)	Model çıktısı üretir	Class veya regression tahmini
predict_proba(X)	Class probability üretir	Classification

Leakage sezgisi

Validation set üzerinde **fit** yapılmamalıdır. Validation yalnız training fold üzerinde öğrenilmiş dönüşümle **transform** edilmelidir.

```
# Yanlış zihinsel model
scaler.fit(X_all)          # validation bilgisi sızabilir
X_scaled = scaler.transform(X_all)

# Doğru zihinsel model
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_valid_scaled = scaler.transform(X_valid)
```

2.1 Stateful vs stateless transformation

Tür	Örnek	Fit gerekir mi?
Stateful	StandardScaler, SimpleImputer, OneHotEncoder	Evet
Stateless	Sabit matematiksel dönüşüm, bazı FunctionTransformer fonksiyonları	Hayır veya trivial
Estimator	LogisticRegression, RandomForest	Evet

3. sklearn Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=2000))
])

pipe.fit(X_train, y_train)
proba = pipe.predict_proba(X_valid)[:, 1]
```

Pipeline içindeki son adım estimator olabilir. Önceki adımların transform metoduna sahip olması gerekir.

Adım adı	Neden önemli?
impute	Eksik değer politikasını model artifact içine alır.
scale	Training mean/std değerlerini saklar.
model	Preprocessed veride parametre öğrenir.

predict sırasında ne olur?

Pipeline yeni X geldiğinde otomatik olarak impute -> scale -> model.predict sırasını çalıştırır. Ayrı preprocessing kodu yazmana gerek kalmaz.

3.1 make_pipeline

```
from sklearn.pipeline import make_pipeline

pipe = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    LogisticRegression(max_iter=2000)
)
```

`make_pipeline` otomatik step isimleri verir; kısa prototiplerde kullanışlıdır. Tuning ve okunabilirlik için açık isimli Pipeline çoğu zaman daha rahattır.

4. ColumnTransformer - farklı sütunlara farklı işlemler

Gerçek tabular veride numeric, categorical ve bazen text sütunlar aynı preprocessing ihtiyacına sahip değildir. ColumnTransformer bu dalları tek graph içinde birleştirir.

Numeric → Imputer → Scaler

Categorical → Imputer → OneHot

Text → TF-IDF

Birleştir → Model

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

num_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler())
])

cat_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

prep = ColumnTransformer([
    ("num", num_pipe, numeric_cols),
    ("cat", cat_pipe, categorical_cols)
])
```

handle_unknown="ignore"

Production'da training sırasında görülmeyen kategori gelebilir. Bu seçenek inference'ın patlamasını önler; ancak unseen-rate ayrıca izlenmelidir.

5. Uçtan uca tabular classification pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("prep", prep),
    ("clf", LogisticRegression(
        class_weight="balanced",
        max_iter=2000
    ))
])

model.fit(X_train, y_train)
proba = model.predict_proba(X_valid)[:, 1]
```

Kazanç	Açıklama
Tek fit	Imputer, encoder, scaler ve model aynı training verisinde fit olur.
Tek predict	Yeni veri aynı preprocess akışından geçer.
CV güvenliği	Her fold kendi preprocess parametrelerini öğrenir.
Serialization	Tek artifact kaydedilebilir.

Senior bakış

"Model" dediğinde yalnız classifier'ı değil, preprocessing + feature logic + estimator bütününü düşün.

6. Cross-validation neden pipeline içinde yapılmalı?

Preprocessing önce tüm dataset üzerinde yapılırsa validation fold'unun istatistikleri training fold'una sızabilir. Pipeline kullanıldığında scikit-learn her fold için adımları sıfırdan fit eder.

```
from sklearn.model_selection import StratifiedKFold, cross_validate

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

scores = cross_validate(
    model,
    X,
    y,
    cv=cv,
    scoring={
        "roc_auc": "roc_auc",
        "pr_auc": "average_precision"
    }
)
```

Fold mantığı

Her iterasyonda: train fold -> prep.fit_transform -> model.fit; validation fold -> prep.transform -> model.predict. Validation hiçbir preprocessing parametresinin öğrenilmesine katılmaz.

6.1 CV splitter da pipeline kadar önemlidir

Veri yapısı	Splitter
IID classification	StratifiedKFold
Regression / genel IID	KFold
Aynı customer birden çok satır	GroupKFold
Zaman bağımlı	TimeSeriesSplit / temporal folds

7. Pipeline ile hyperparameter tuning

Pipeline içindeki adımlara `step__parameter` syntax'ı ile erişilir.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "prep__num__impute__strategy": ["median", "mean"],
    "clf__C": [0.01, 0.1, 1, 10],
    "clf__class_weight": [None, "balanced"]
}

search = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    scoring="average_precision",
    cv=cv,
    n_jobs=-1
)

search.fit(X_train, y_train)
print(search.best_params_)
```

Neden güçlü?

Preprocessing seçimi ile model hyperparameter seçimi aynı validation mantığı içinde optimize edilir.

7.1 RandomizedSearch ve Optuna

Arama uzayı büyüdükçe GridSearch maliyetli olur. RandomizedSearch veya Optuna gibi yöntemlerde de pipeline estimator olarak kullanılır.

8. passthrough, drop ve remainder

```
prep = ColumnTransformer(
    transformers=[
        ("num", num_pipe, numeric_cols),
        ("cat", cat_pipe, categorical_cols)
    ],
    remainder="drop"   # veya "passthrough"
)
```

Seçenek	Davranış
drop	Listelenmeyen sütunları atar.
passthrough	Listelenmeyen sütunları olduğu gibi geçirir.

Şema disiplini

Production sisteminde istemeden yeni bir sütunu passthrough ile modele taşımak risklidir. Explicit feature listesi daha güvenli olabilir.

8.1 Feature leakage örneği

DataFrame'e sonradan `churn_reason` gibi target sonrası bir kolon eklendi ve `remainder="passthrough"` kullanıldıysa pipeline farkında olmadan leak eden sütunu modele sokabilir.

9. FunctionTransformer ve custom feature logic

```
import numpy as np
from sklearn.preprocessing import FunctionTransformer

log_transform = FunctionTransformer(
    np.log1p,
    feature_names_out="one-to-one"
)
```

Basit stateless fonksiyonlar için FunctionTransformer uygundur. Daha karmaşık dönüşümlerde custom transformer yazılır.

```
from sklearn.base import BaseEstimator, TransformerMixin

class RatioFeatures(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        X["avg_order_value"] = (
            X["revenue"] / X["orders"].clip(lower=1)
        )
        return X
```

Custom transformer contract

fit -> self döndür; transform -> aynı schema mantığıyla yeni X üret. mümkünse fit içinde training-data'dan öğrenilen state'i sakla.

10. Feature names ve DataFrame çıktısı

Pipeline debugging sırasında hangi feature'ların modele girdiğini görmek önemlidir.

```
# sklearn sürümüne bağlı olarak
prep.set_output(transform="pandas")

X_transformed = prep.fit_transform(X_train)
print(X_transformed.columns)
```

```
# OneHotEncoder sonrası isimler
feature_names = prep.get_feature_names_out()
```

Neden önemli?

Feature importance, coefficient analizi, SHAP ve schema testleri için final feature isimleri gerekir.

10.1 Sparse vs dense output

Durum	Tercih
Çok sayıda one-hot kategori	Sparse matrix memory açısından avantajlı
Bazı estimatorlar sparse kabul etmiyor	Dense dönüşüm gerekebilir
High-cardinality	One-hot yerine farklı encoding/model düşün

11. Feature selection pipeline içinde

```
from sklearn.feature_selection import SelectKBest, mutual_info_classif

pipe = Pipeline([
    ("prep", prep),
    ("select", SelectKBest(mutual_info_classif, k=50)),
    ("model", LogisticRegression(max_iter=2000))
])
```

Neden pipeline içinde?

Feature selection tüm dataset üzerinde yapılırsa validation target bilgisi training'e sızabilir. Selection da her CV fold içinde yeniden yapılmalıdır.

11.1 Model-based selection

L1 logistic, tree importance veya permutation-based seçim kullanılabilir; ancak selection kriteri de validation düzenine dahil edilmelidir.

12. Imbalanced data pipeline

Oversampling/SMOTE gibi adımlar standart sklearn Pipeline yerine çoğunlukla `imblearn.pipeline.Pipeline` ile uygulanır; çünkü sampling yalnız training üzerinde yapılmalıdır.

```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE

imb_pipe = ImbPipeline([
    ("prep", prep),
    ("smote", SMOTE(random_state=42)),
    ("model", LogisticRegression(max_iter=2000))
])
```

Kritik leakage

SMOTE'u CV'den önce tüm dataset üzerinde yapmak validation gözlemlerine dayalı sentetik training örnekleri yaratabilir.

12.1 Alternatif

Class weights ve threshold tuning çoğu problemde sampling'den önce değerlendirilmesi gereken güçlü ve daha basit araçlardır.

13. Regression pipeline örneği

```
from sklearn.linear_model import Ridge

reg_pipe = Pipeline([
    ("prep", prep),
    ("model", Ridge(alpha=1.0))
])

reg_pipe.fit(X_train, y_train)
pred = reg_pipe.predict(X_test)
```

Regression adımı	Pipeline faydası
Imputation	Validation median/mean sızmaz
One-hot	Unseen category yönetilir
Scaling	Ridge/Lasso penalty adil çalışır
Model	Aynı artifact ile inference

Target transformation

Target için log dönüşümü gerekiyorsa TransformedTargetRegressor kullanılabilir; bu dönüşüm feature pipeline'ından ayrı ama aynı estimator graph'ının parçası olur.

```
from sklearn.compose import TransformedTargetRegressor

model = TransformedTargetRegressor(
    regressor=reg_pipe,
    func=np.log1p,
    inverse_func=np.expm1
)
```

14. Train-serving skew

Training sırasında feature bir şekilde, production inference sırasında başka şekilde hesaplanırsa model aynı isimli ama farklı anlamdaki veriyi görür.

Skew örneği	Sonuç
Train: median impute, serving: 0 fill	Distribution değişir
Train: lower-case category, serving: raw	Unknown category artar
Train: UTC timestamp, serving: local time	Time feature bozulur
Train: 30d aggregate, serving: 28d query	Feature semantics değişir

Çözüm

Mümkün olduğunca training ve inference için aynı transformation code path'ını kullan. Feature logic'i notebook içinde kopyalama.

14.1 Batch ve online inference

Aynı pipeline artifact batch scoring scriptinde, API endpoint'inde veya scheduled job'da kullanılabilir. Online sistemde latency ve dependency maliyeti ayrıca ölçülmelidir.

15. Serialization - pipeline kaydetme

```
import joblib

joblib.dump(model, "churn_pipeline.joblib")

loaded = joblib.load("churn_pipeline.joblib")
proba = loaded.predict_proba(new_data)[:, 1]
```

Uyarı

Pickle/joblib dosyalarını yalnız güvenilir kaynaktan yükle. Ayrıca Python ve library versiyonları model artifact uyumluluğunu etkileyebilir.

15.1 Model artifact yanında ne saklanmalı?

Artifact	Neden
pipeline.joblib	Dönüşüm + model
requirements/lockfile	Library versiyonları
feature schema	Input kolonları ve tipleri
training metadata	Dataset snapshot, tarih, git commit
metrics	Validation/test performansı
threshold/config	Prediction sonrası karar mantığı

16. Schema validation

Pipeline yalnız değerleri değil giriş şemasını da bekler. Production'da eksik kolon, yanlış dtype veya beklenmeyen kategori model çağrısını bozabilir.

Kontrol	Örnek
Required columns	customer_id dışındaki feature kolonları var mı?
dtype	age numeric mi?
range	age 0-120 aralığında mı?
null rate	income missing oranı aşırı arttı mı?
category domain	plan_type beklenen değerlerde mi?

Pipeline + validation

sklearn Pipeline preprocessing yapar; schema/data contract doğrulaması için Pandera, Pydantic veya custom validation katmanı ayrıca kullanılabilir.

16.1 Fail fast

Yanlış şemayı sessizce modele sokmak yerine scoring öncesi açık hata vermek çoğu production sisteminde daha güvenlidir.

17. Pipeline testleri

Test	Amaç
Fit smoke test	Küçük örnekte pipeline fit oluyor mu?
Predict shape	Çıktı satır sayısı input ile eşleşiyor mu?
Missing category	Unseen kategori inference'ı bozuyor mu?
Missing value	NaN akışı çalışıyor mu?
Schema test	Zorunlu kolon eksikse hata veriyor mu?
Determinism	random_state ile tekrar üretilebilir mi?

```
def test_prediction_shape(trained_pipeline, sample_df):
    pred = trained_pipeline.predict(sample_df)
    assert len(pred) == len(sample_df)


def test_unknown_category(trained_pipeline, sample_df):
    x = sample_df.copy()
    x.loc[x.index[0], "device"] = "brand_new_device"
    trained_pipeline.predict(x)  # exception olmamalı
```

Unit test hedefi

ML metric'i unit test ile garanti edemezsin; ama data contract ve transformation behavior'ı test edebilirsin.

18. Reproducibility

Aynı kodu tekrar çalıştırdığında aynı sonucu veya açıklanabilir küçük farkları üretebilmek profesyonel ML sisteminin temelidir.

Katman	Yapılacak
Randomness	random_state / seed
Code	Git commit / tag
Environment	Pinned dependency versions
Data	Dataset snapshot / query version
Config	Hyperparameters ayrı config
Artifact	Pipeline version / model registry

Pipeline tek başına yetmez

Pipeline preprocessing'i kapsar; veri snapshot ve environment versioning ayrıca gerekir.

18.1 random_state her şeyi çözmez

Bazı paralel/GPU işlemleri deterministik olmayabilir. Production-grade reproducibility için framework seviyesinde ek ayarlar gerekebilir.

19. Caching ve performans

Ağır preprocessing adımları GridSearch sırasında tekrar tekrar çalışıyorsa Pipeline memory cache kullanılabilir.

```
from joblib import Memory
from sklearn.pipeline import Pipeline

memory = Memory(location="./cache", verbose=0)

pipe = Pipeline(
    steps=[
        ("prep", prep),
        ("model", LogisticRegression())
    ],
    memory=memory
)
```

Ne zaman yararlı?

Preprocessing pahalı ve model hyperparameter'ları sık değişiyorsa. Çok küçük datasetlerde cache overhead gereksiz olabilir.

19.1 n_jobs

Cross-validation ve search katmanında paralellik artırılabilir. Ancak nested parallelism CPU/RAM kullanımını patlatabilir.

20. Pipeline debugging

Tek nesne olması debugging'i zorlaştırmaz; doğru introspection araçlarıyla her adım incelenebilir.

```
print(model.named_steps.keys())

prep_fitted = model.named_steps["prep"]
clf_fitted = model.named_steps["clf"]

print(clf_fitted.coef_)
print(prep_fitted.get_feature_names_out())
```

20.1 Ara çıktıyı kontrol et

```
X_train_transformed = (
    model.named_steps["prep"]
         .transform(X_train.head(20))
)
print(X_train_transformed.shape)
```

Debugging soruları

Kaç feature çıktı? NaN kaldı mı? Sparse/dense beklediğin gibi mi? Category isimleri doğru mu? Scale aralıkları mantıklı mı?

21. Common pipeline anti-patterns

Anti-pattern	Neden kötü?
Scaler'ı CV öncesi fit etmek	Validation leakage
pd.get_dummies train ve test ayrı çalıştırmak	Kolon setleri uyuşmayabilir
Notebook feature kodunu API'de yeniden yazmak	Train-serving skew
Target encoding pipeline dışında	Target leakage riski
Test set ile preprocessing kararı değiştirmek	Test contamination
remainder="passthrough" + kontrolsüz kolonlar	Leak eden/yeni kolon yanlışlıkla modele girebilir
Pipeline'ı kaydedip schema kaydetmemek	Inference contract belirsiz
Random seed yok	Deney tekrarı zor

Kural

Bir dönüşüm modelin gördüğü sayıları değiştiriyorsa, model sisteminin parçasıdır ve mümkün olduğunca pipeline/graph içinde yönetilmelidir.

22. Gerçek vaka: churn pipeline

Bir abonelik şirketi 30 gün içinde churn edecek müşterileri tahmin etmek istiyor.

Katman	Tasarım
Input	age, plan, tenure, sessions_30d, payment_failures_90d, country
Numeric	median impute + StandardScaler
Categorical	most_frequent + OneHotEncoder
Model	LogisticRegression / GradientBoosting benchmark
CV	Temporal veya Group-aware tasarım
Metric	PR-AUC + recall@capacity
Threshold	Retention ekip kapasitesine göre
Artifact	prep + model tek pipeline

```
churn_pipeline = Pipeline([
    ("prep", ColumnTransformer([
        ("num", num_pipe, num_cols),
        ("cat", cat_pipe, cat_cols)
    ])),
    ("model", LogisticRegression(
        class_weight="balanced",
        max_iter=2000
    ))
])
```

Production akışı

API sadece ham schema'yı alır. Pipeline preprocessing ve model skorunu üretir. Threshold/config ayrı karar katmanı olarak versiyonlanabilir.

23. Gerçek vaka: house price regression pipeline

Feature grubu	Pipeline adımı
m2, room_count, age	median impute + scale
district, building_type	one-hot
target price	TransformedTargetRegressor log1p
model	Ridge / ElasticNet
CV	Temporal holdout + KFold on train
metric	MAE + RMSE

Neden iyi örnek?

Numeric + categorical preprocessing, target transformation ve regularized regression aynı sistem içinde birleşir.

23.1 Tuning

```
params = {
    "regressor__model__alpha": np.logspace(-3, 3, 20),
    "regressor__prep__num__impute__strategy": ["median", "mean"]
}
```

24. Pipeline tasarım checklist

1. Prediction time ve input schema net mi?
2. Numeric, categorical, text ve datetime sütunlar ayrıldı mı?
3. Her stateful preprocessing yalnız training fold üzerinde fit oluyor mu?
4. Unknown category ve missing value davranışı tanımlı mı?
5. Feature selection/encoding CV içinde mi?
6. Group/time leakage için doğru splitter var mı?
7. Hyperparameter search step__param formatıyla pipeline üzerinde mi?
8. Test set yalnız final evaluation için mi?
9. Pipeline serialization ve dependency versions kaydediliyor mu?
10. Train-serving skew için aynı transformation code path kullanılıyor mu?
11. Schema validation ve smoke testler var mı?
12. Production metric ve drift monitoring planı var mı?

Pipeline olgunluk testi

Yeni bir satırı ham formatta verip tek bir `predict()` veya `predict_proba()` çağrısıyla güvenilir çıktı alabiliyor musun?

25. Mülakat soruları

Soru	Kısa cevap
Pipeline neden kullanılır?	Preprocessing + model akışını reproducible ve leakage-safe yapmak için.
fit ile transform farkı?	fit parametre öğrenir; transform öğrenilmiş dönüşümü uygular.
ColumnTransformer ne yapar?	Farklı sütun gruplarına farklı preprocessing uygular ve çıktıları birleştirir.
Scaler neden CV içinde olmalı?	Validation mean/std bilgisi training'e sızmasın diye.
OneHotEncoder handle_unknown neden?	Yeni kategoride inference hata vermesin diye.
GridSearch pipeline parametre syntax?	step__parameter.
Feature selection pipeline dışında olursa risk?	Validation target/data leakage.
SMOTE neden imblearn Pipeline içinde?	Sadece training fold üzerinde uygulanması için.
Train-serving skew nedir?	Training ve serving feature hesaplarının farklı olması.
Pipeline nasıl kaydedilir?	joblib/pickle; versiyon ve schema ile birlikte.
remainder passthrough riski?	İstenmeyen/leak eden yeni kolonlar modele girebilir.
Custom transformer nasıl yazılır?	BaseEstimator + TransformerMixin, fit/transform contract.
Pipeline debugging nasıl?	named_steps, transform, get_feature_names_out.
Reproducibility için pipeline yeterli mi?	Hayır; data, environment, code, config de versionlanmalı.
Schema validation pipeline'ın işi mi?	Kısmen preprocessing; ama explicit data contract katmanı genellikle ayrıca gerekir.

26. Uygulama alıştırmaları

1. Numeric + categorical bir dataset için ColumnTransformer kur.
2. Aynı modeli preprocessing pipeline dışında ve içinde CV ile karşılaştır; leakage farkını gözlemle.
3. OneHotEncoder handle_unknown kapalıyken unseen kategori ile inference dene.
4. GridSearchCV ile imputer strategy ve LogisticRegression C parametresini birlikte tune et.
5. Custom transformer ile avg_order_value feature üret.
6. SelectKBest adımını pipeline içine ekleyip CV skorunu ölç.
7. GroupKFold ile müşteri tekrarları olan veride pipeline değerlendir.
8. TimeSeriesSplit ile scaler leakage'ı engelle.
9. joblib ile pipeline kaydet-yükle ve prediction parity test et.
10. Input schema eksik kolon geldiğinde fail-fast test yaz.
11. Feature isimlerini çıkarıp logistic coefficient tablosu oluştur.
12. SMOTE pipeline ile outside-SMOTE yaklaşımının CV skor farkını karşılaştır.

Mini portföy projesi

Çıktı	İçerik
src/pipeline.py	ColumnTransformer + estimator factory
src/features.py	Custom transformers
src/schema.py	Input validation
train.py	CV + tuning + artifact export
predict.py	Artifact load + batch scoring
tests/	Schema, unknown category, prediction parity
model.joblib	Versioned pipeline artifact
README.md	Architecture, metrics, reproducibility

CV cümlesi

Built a leakage-safe end-to-end scikit-learn pipeline with column-wise preprocessing, cross-validated hyperparameter tuning, custom feature transformers, schema checks and serialized inference artifact.

27. Final cheat sheet

Kavram	Hatırlanacak fikir
Pipeline	Transformations + estimator tek graph
fit	Training verisinden state öğren
transform	Öğrenilmiş state uygula
ColumnTransformer	Column-specific preprocessing
handle_unknown	Unseen category güvenliği
CV içinde preprocessing	Leakage engelle
step__param	Pipeline tuning syntax
Custom transformer	Reusable feature logic
Feature selection	CV içinde
SMOTE	Training fold içinde
Serialization	Aynı artifact ile inference
Schema validation	Input contract
Train-serving skew	Aynı transformation code path
Reproducibility	Code + data + env + config + artifact
Tests	Pipeline davranışını güvenceye al

Bu modülden sonra

Cross Validation ve Hyperparameter Optimization bölümlerinde model selection süreçlerini daha güvenli ve profesyonel kurabileceksin. Pipeline artık notebook kolaylığı değil, ML sistem tasarımının temel yapı taşı olmalı.